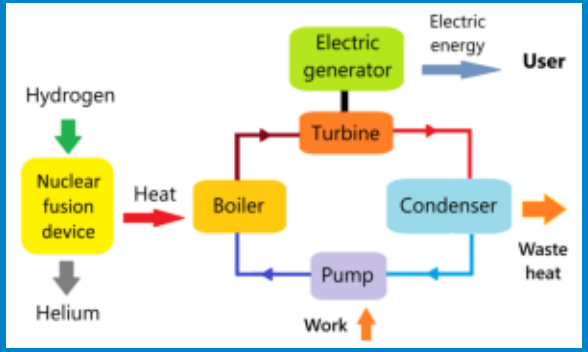
Automated Problem-to-Solution Generation for a Tokamak Fusion-Plasma Simulation



A hierarchical multi-agent PETSc system — applied and verified

PRESENTER

[PRESENTER NAME] / Summer 2026 Intern, Argonne National Laboratory

EVENT

Argonne Summer 2026 Intern Presentation

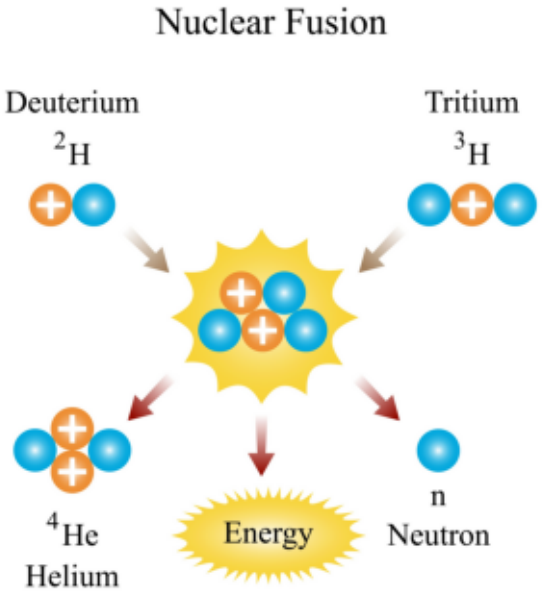
DATE

2026

Why fusion energy needs trustworthy simulation

Fusion could be a near-limitless clean energy source — if we can confine the plasma

- Fusion fuses light nuclei (deuterium + tritium) to release energy — the reaction that powers the Sun.
- A tokamak confines a ~150-million-°C plasma in a magnetic “cage” long enough to fuse.
- Getting there is a simulation problem: predict the plasma before building costly hardware.
- Simulations must be trustworthy — a plausible-looking wrong answer is worse than none.

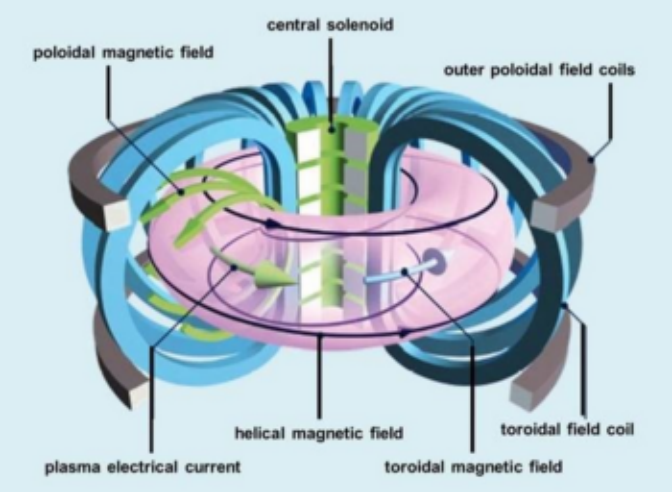


Deuterium–tritium fusion

The modeling challenge

Turning fusion physics into correct, fast HPC code needs scarce, implicit expertise

- Numerical methods: choose grids, discretizations, and solvers that actually converge.
- Libraries & parallelism: express it in PETSc and run on many CPU cores / GPUs.
- Verification: prove the computed answer is right, not merely plausible.
- Question: can a multi-agent AI system automate this whole path — and can we trust it?

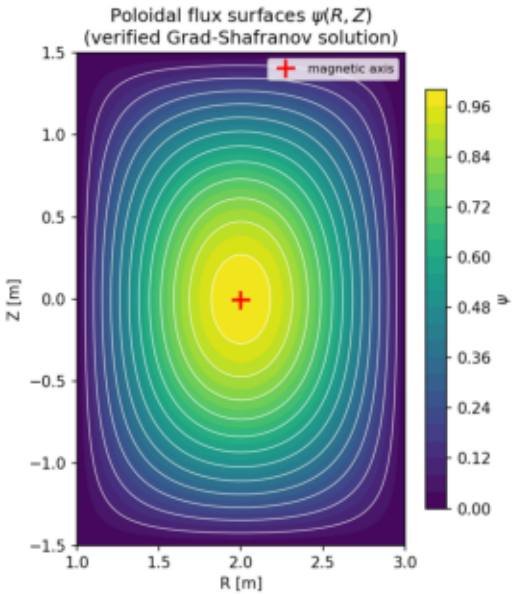


Tokamak magnetic confinement (U.S. DOE)

The problem: Grad–Shafranov equilibrium

Axisymmetric ideal-MHD force balance for the poloidal flux ψ(R,Z)

- $\Delta^*\psi = -\mu_0 R^2\, p'(\psi) - F\, F'(\psi)$ — elliptic, nonlinear, time-independent.
- Its solution gives the flux surfaces, magnetic axis, and safety factor q.
- A natural PETSc SNES (nonlinear solver) problem.
- Verifiable: a manufactured exact solution gives a rigorous convergence test.



Flux surfaces of the verified solution

The system: a 3-layer multi-agent pipeline

PETSc's Model-Context-Protocol (MCP) agents, coordinated end to end

- Problem definition — Mathematical Modeling agent → strong/weak form, name, time-dependence.
- Agent execution — Numerical Analysis agent → grid, discretization, solver (SNES).
- Agent execution — HPC Code Generation agent → writes, compiles & runs PETSc C.
- Execution substrate — compile-run agent (make / run on the real machine).
- All agents run against Argonne's Argo gateway (Claude Opus 4.8).
- A project-owned driver records every artifact with provenance (reproducible, resumable).

The pipeline in action

From one English prompt to a compiled, running solver — with no human edits

- Prompt: “the Grad-Shafranov equilibrium for the plasma in a tokamak.”
- Modeling agent → identified ‘Grad-Shafranov equation’; time-dependent = False.
- Numerical Analysis agent → nonlinear solve with SNES on a structured grid.
- Code Generation agent → 267-line PETSc program (DMDA + SNES + true Jacobian).
- Compiled and ran on 1 and 4 MPI ranks — no human edits.

The agent-generated PETSc solver (excerpt)

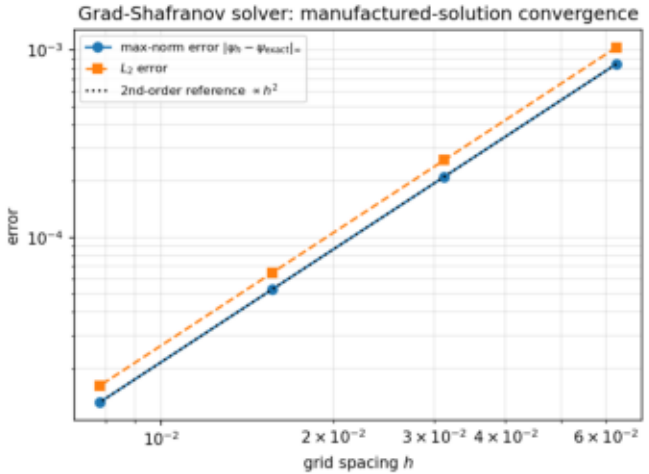
A true-Jacobian finite-difference SNES residual — written by the Code Generation agent

```
static PetscErrorCode FormFunction(SNES snes, Vec X, Vec F, void *ctx){
  /* Delta*_h psi = (psi_E-2psi_C+psi_W)/hR^2
                  - (1/R)(psi_E-psi_W)/(2hR)
                  + (psi_N-2psi_C+psi_S)/hZ^2   */
  f[j][i] = dRR - dR1/R + dZZ - ForcingF(user,R,Z);   /* residual */
}
SNESSetFunction(snes, r, FormFunction, &user);
SNESSetJacobian(snes, J, J, FormJacobian, &user);   /* true Jacobian */
SNESSolve(snes, NULL, x);
```

Verification: it is actually right

Method of manufactured solutions — a measured order of accuracy, not just plausible output

- Known exact ψ → check the numerical error as the grid refines.
- Max-norm error = 1.32e-05 on the finest 257² grid.
- Observed order of accuracy p = 2.00, 2.00, 2.00 (expected 2 for central differences).
- SNES converged: CONVERGED_FNORM_RELATIVE.



Manufactured-solution convergence

Decision-gate metrics

Correctness, human effort, and efficiency — the numbers behind the demo

- Correctness: model identified ✓, compiled & ran ✓, 2nd-order convergence ✓.
- Human effort: 0 lines of solver code hand-written — it was generated.
- Efficiency: total wall-clock ≈ 450.8 s; code-gen used 5 tool calls.
- ≈ 23 LLM completions across the whole pipeline.

Fully autonomous: the built-in orchestrator

The shipped LLM orchestrator drives all four agents itself — no Python driver

- Given only the prompt, an inner Claude agent sequenced all four MCP servers on its own.
- It independently chose a Solov’ev Grad–Shafranov model and a DMPLEX + PetscFE finite-element solve.
- The generated code compiled cleanly and ran: 225 DOFs, L2 norm ψ = 2.04, return code 0.
- It surfaced a real upstream bug — a hardcoded iteration cap flagged a converged run as failure.
- We fixed it and contributed the patch; the re-run finished: “I have completed the orchestration.”

What I built and contributed

Reusable tooling around the agents, plus fixes contributed back upstream

- A project-owned orchestration driver with full artifact provenance (resumable).
- Verification + post-processing (convergence, flux surfaces) and a metrics harness.
- Four fixes contributed back to the open PETSc multi-agent system:
  - CWD-independent server resolution; portable stdio interpreter
  - graceful operation without the documentation / RAG services
  - code-generation and orchestrator loops that reliably capture results

Takeaways & next steps

A verification-driven multi-agent path from prompt to trustworthy HPC code

- A multi-agent system generated a correct, verified PETSc fusion-MHD solver from a prompt.
- Both the Python driver and the fully-autonomous orchestrator succeeded end to end.
- Verification-driven: convergence + exact-solution checks, not just plausible output.
- Next: shaped real-machine equilibria (Solov’ev / Cerfon–Freidberg), q-profile, GPU scale-up.
- Code + docs: github.com/engineer-scientist/petsc_mcp_servers_tokamak

Argonne National Laboratory

Thank you